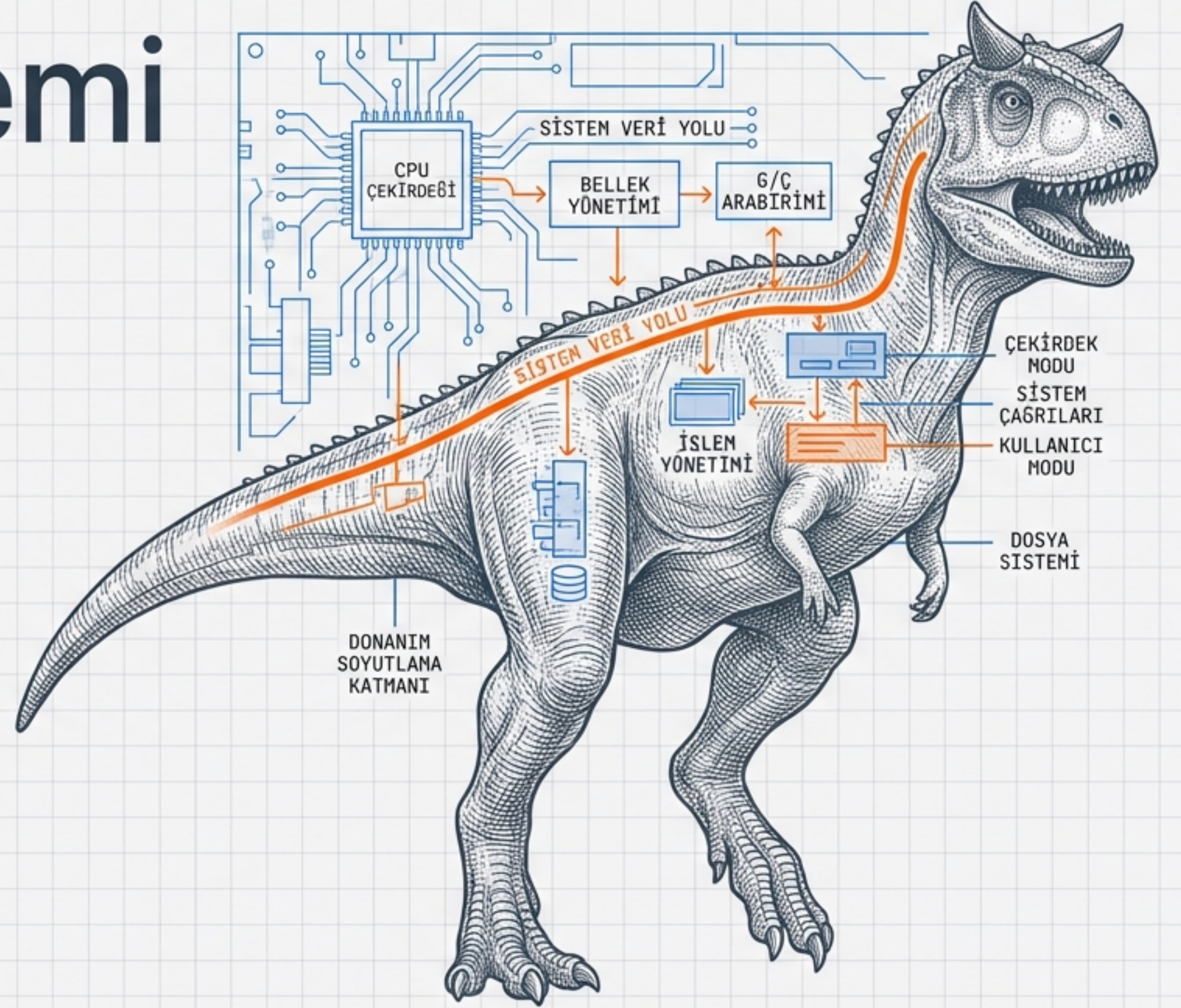


İşletim Sistemi Yapıları

Hizmetler, Sistem Çağrıları ve Mimari Tasarım



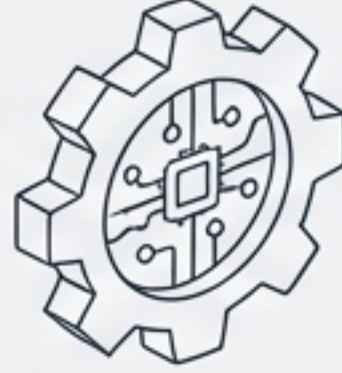
Bölüm 2: Tasarım ve Uygulama İlkeleri

İşletim Sistemi Hizmetleri: İkili Bir Bakış

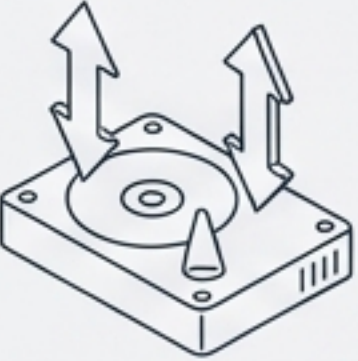
Kullanıcı Odaklı Hizmetler



Kullanıcı Arayüzü (UI)
(CLI, GUI, Dokunmatik)

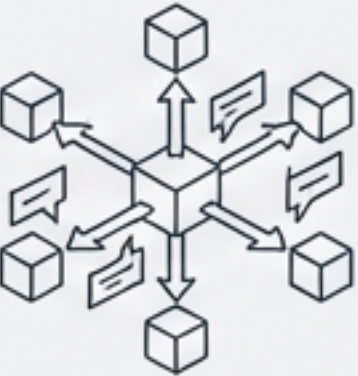
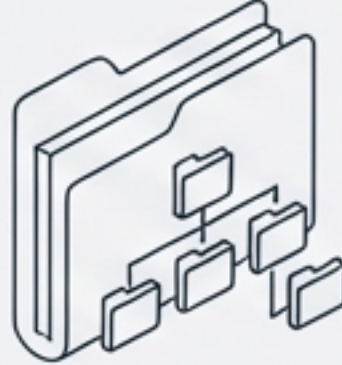


Program Yürütme



G/Ç İşlemleri

Dosya Sistemi

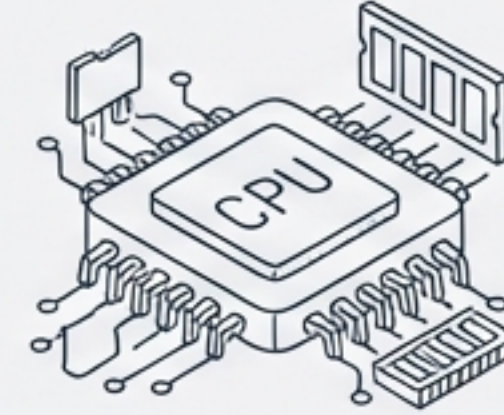


İletişim

Hata Tespiti



Sistem Verimliliği Odaklı



Kaynak Tahsisi
(CPU, Bellek)



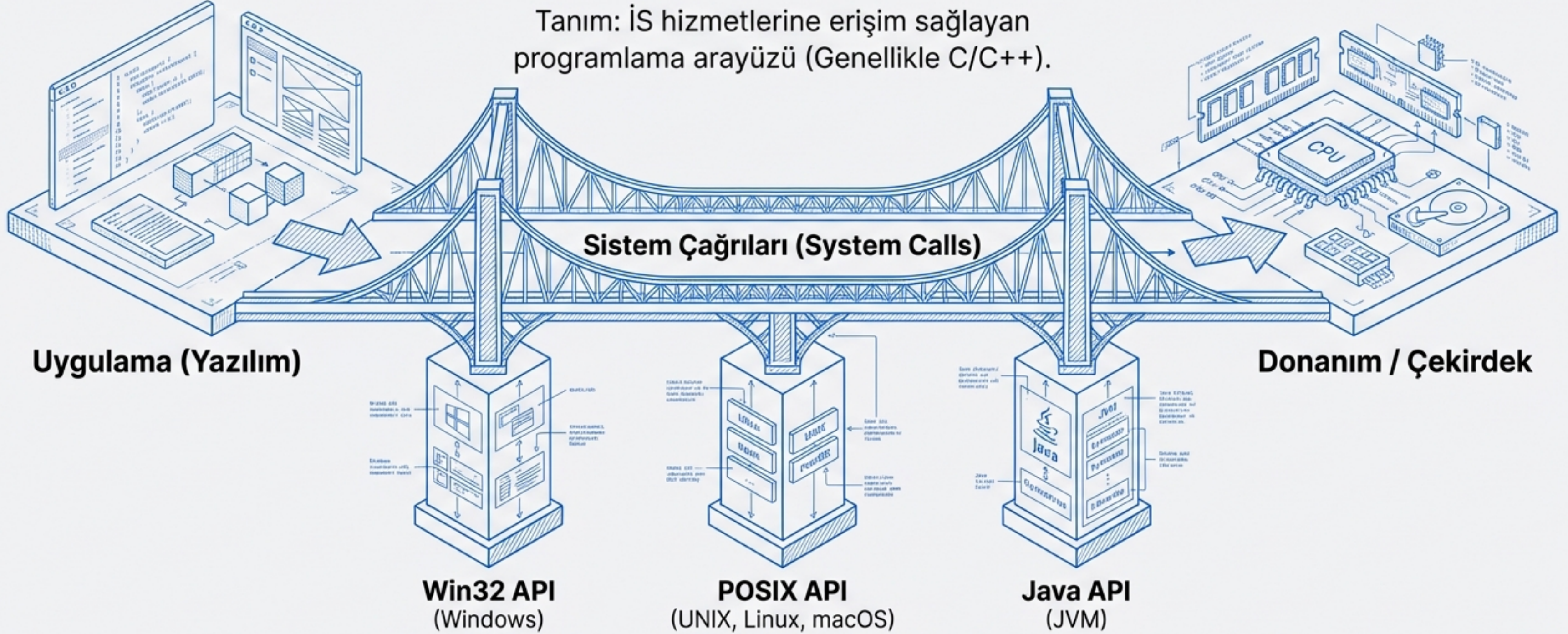
Muhasebe
(Kullanım Takibi)



**Koruma ve
Güvenlik**

Sistem Çağrıları: Yazılım ve Donanım Arasındaki Köprü

Tanım: İS hizmetlerine erişim sağlayan programlama arayüzü (Genellikle C/C++).



Programcılar doğrudan donanımla konuşmaz, API kullanır.

Bir Sistem Çağrısının Anatomisi

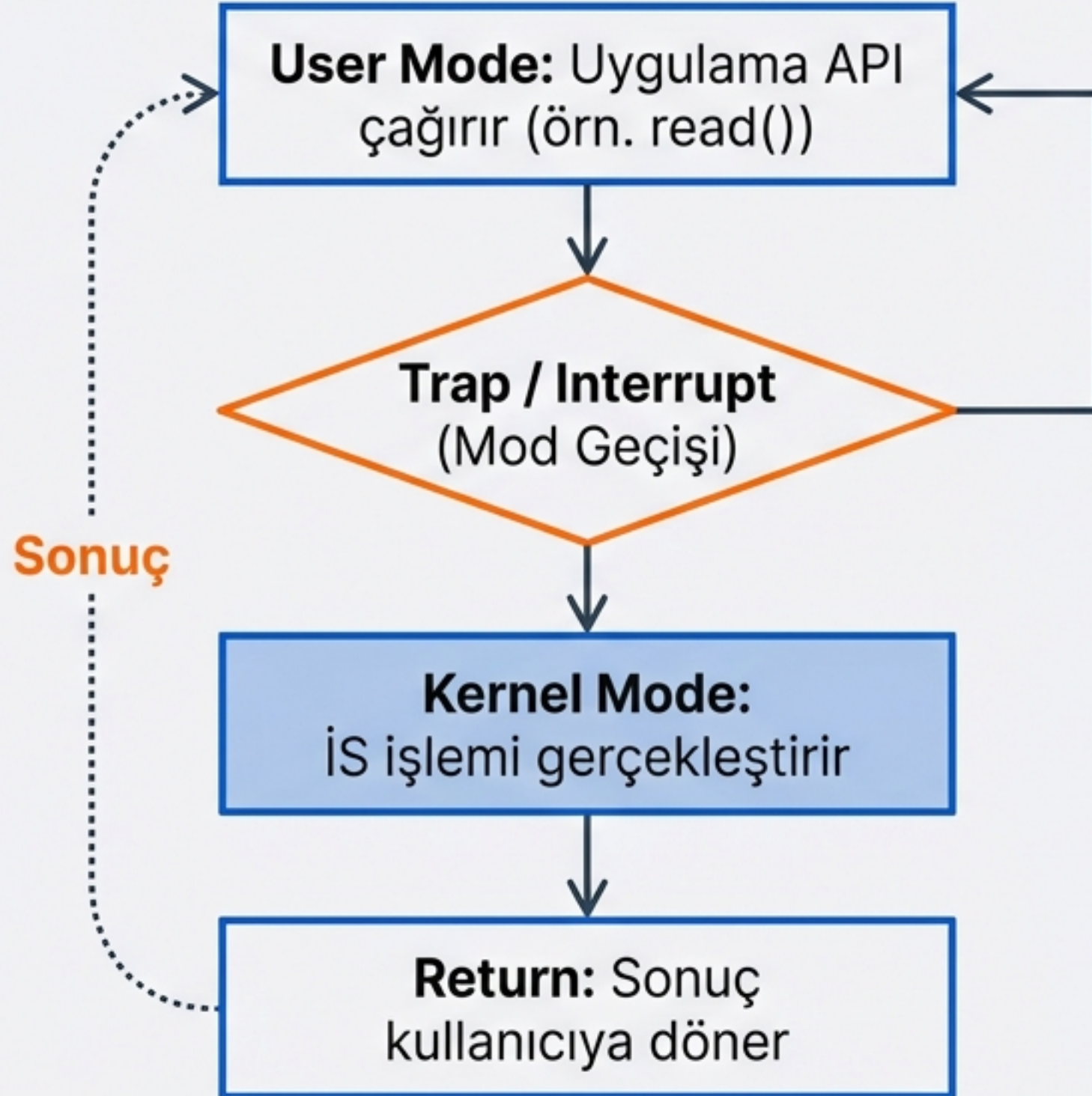
```
#include <unistd.h>
```

```
ssize_t read(int fd, void *buf, size_t count)
```

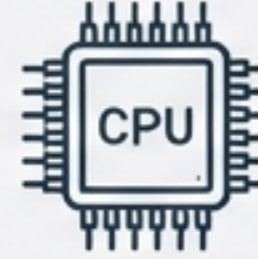
return
value

function
name

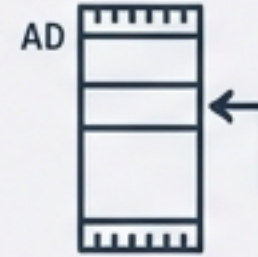
parameters



Parametre Aktarım Yöntemleri



Registers: En basit yöntem, parametreler işlemci yazmaçlarında.

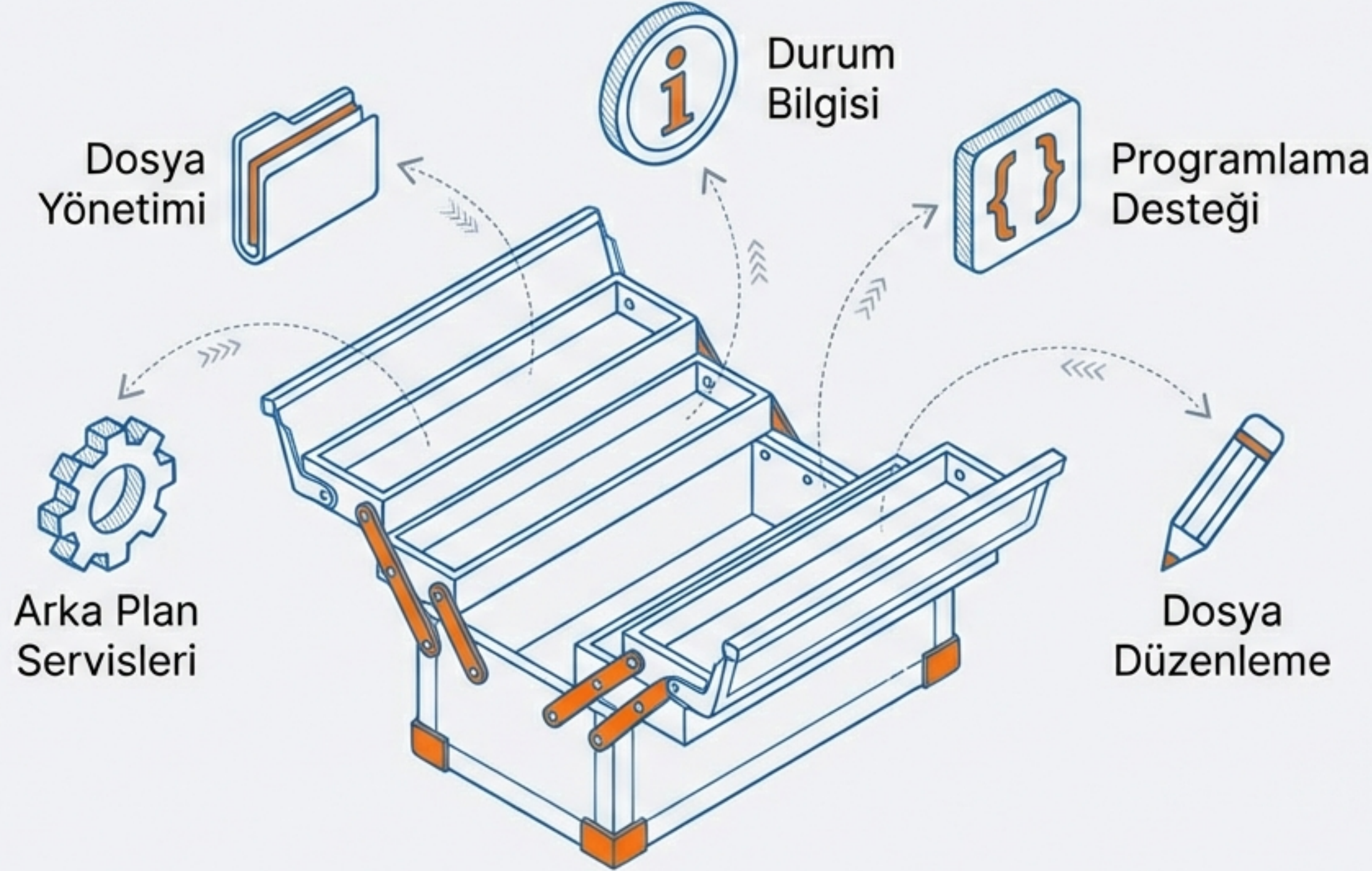


Block/Table: Bellek bloğu adresi iletilir (Linux/Solaris).



Stack: Parametreler yığına itilir (Push), İS çeker (Pop).

Sistem Programları: Kullanıcının Alet Çantası



Kullanıcıların 'İşletim Sistemi' olarak gördüğü çoğu bileşen aslında çekirdek değil, **sistem programlarıdır**.

Tasarım ve Uygulama İlkeleri



NASIL yapılır?
(Örn: Zamanlayıcı donanımı)

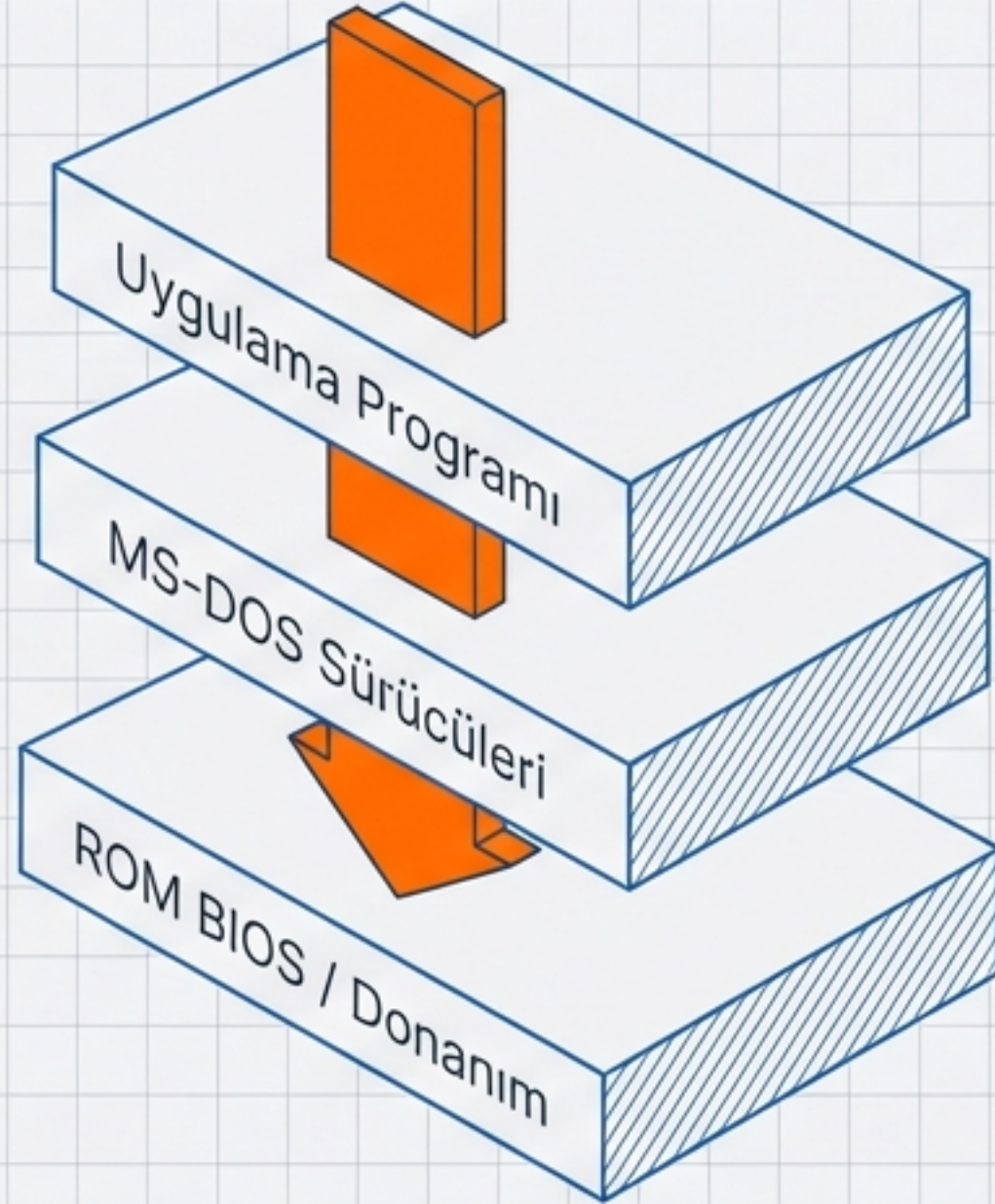
İlke: Mekanizma ve Politikanın ayrılması, sisteme maksimum esneklik sağlar.

NE yapılacak?
(Örn: Kime ne kadar süre verilecek?)

Modern Uygulama: Assembly yerine C/C++ kullanımı taşınabilirliği (Portability) artırır.

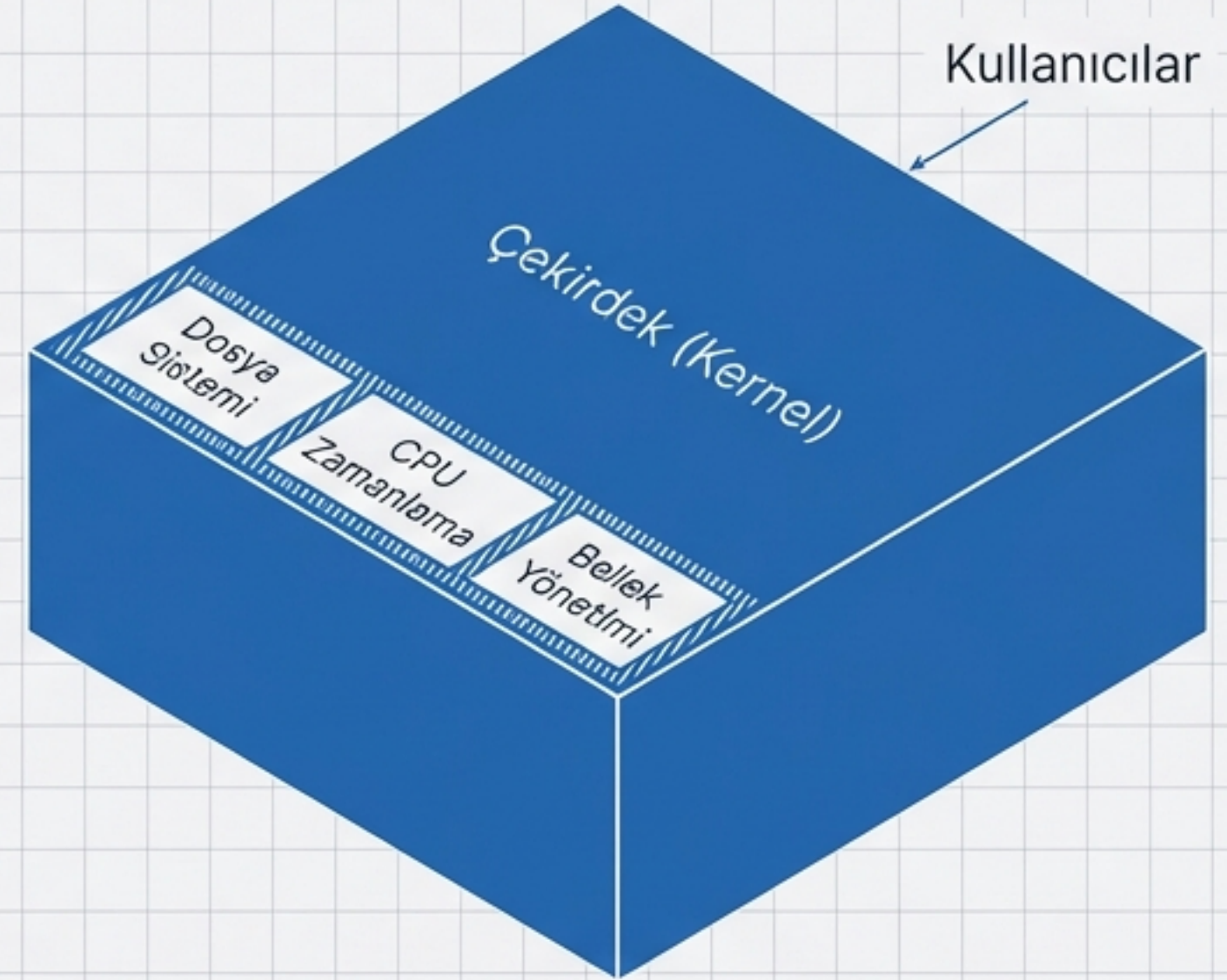
İS Yapıları: Basit ve Monolitik

Basit Yapı (MS-DOS)



Güvensiz: Doğrudan donanım erişimi.

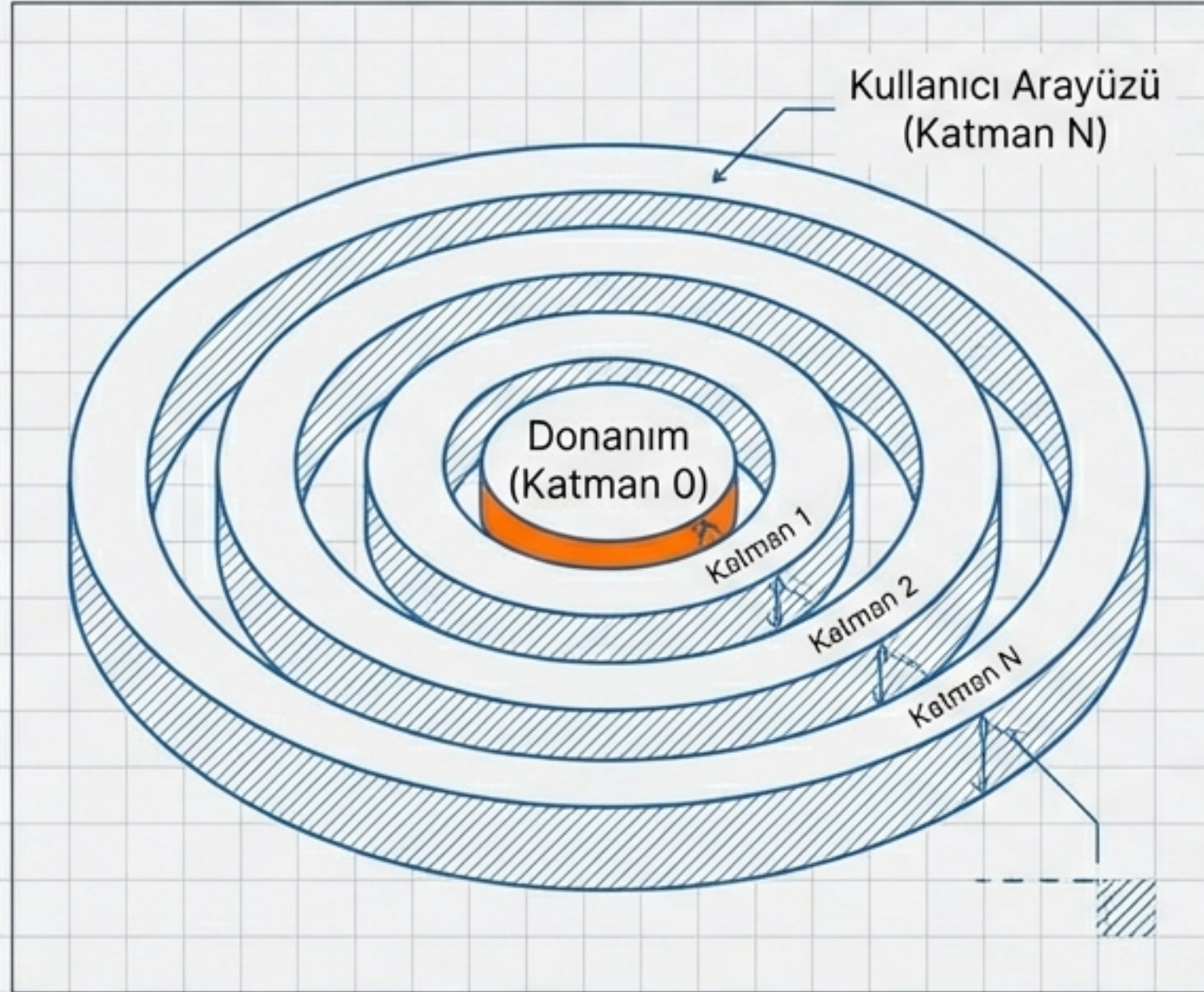
Monolitik Yapı (Orijinal UNIX)



Performanslı ama karmaşık. Tek parça, yönetimi zor.

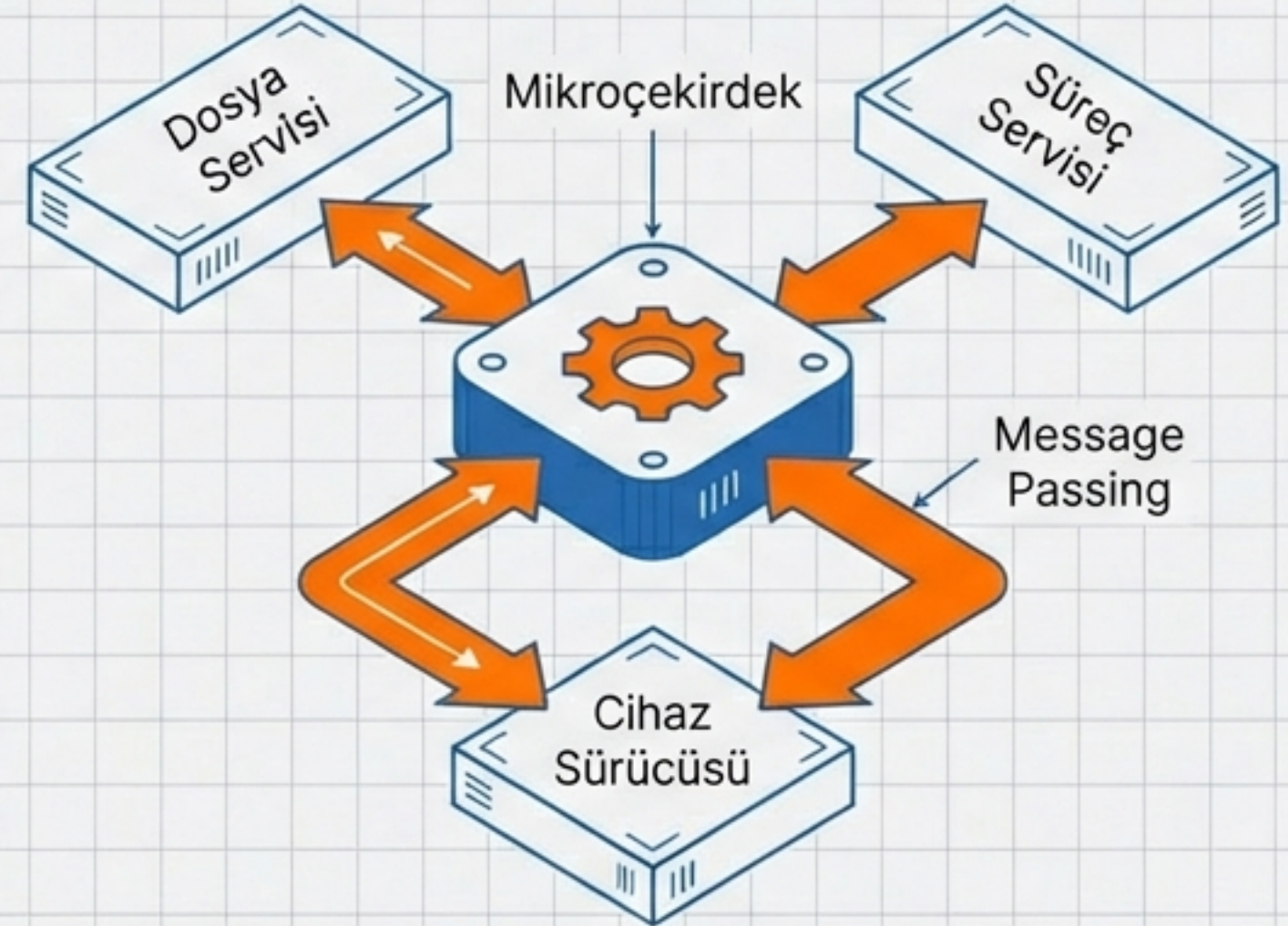
İS Yapıları: Katmanlı ve Mikroçekirdek

Katmanlı Yaklaşım (Layered)



↳ Hata ayıklama kolay, her katman sadece altını kullanır.

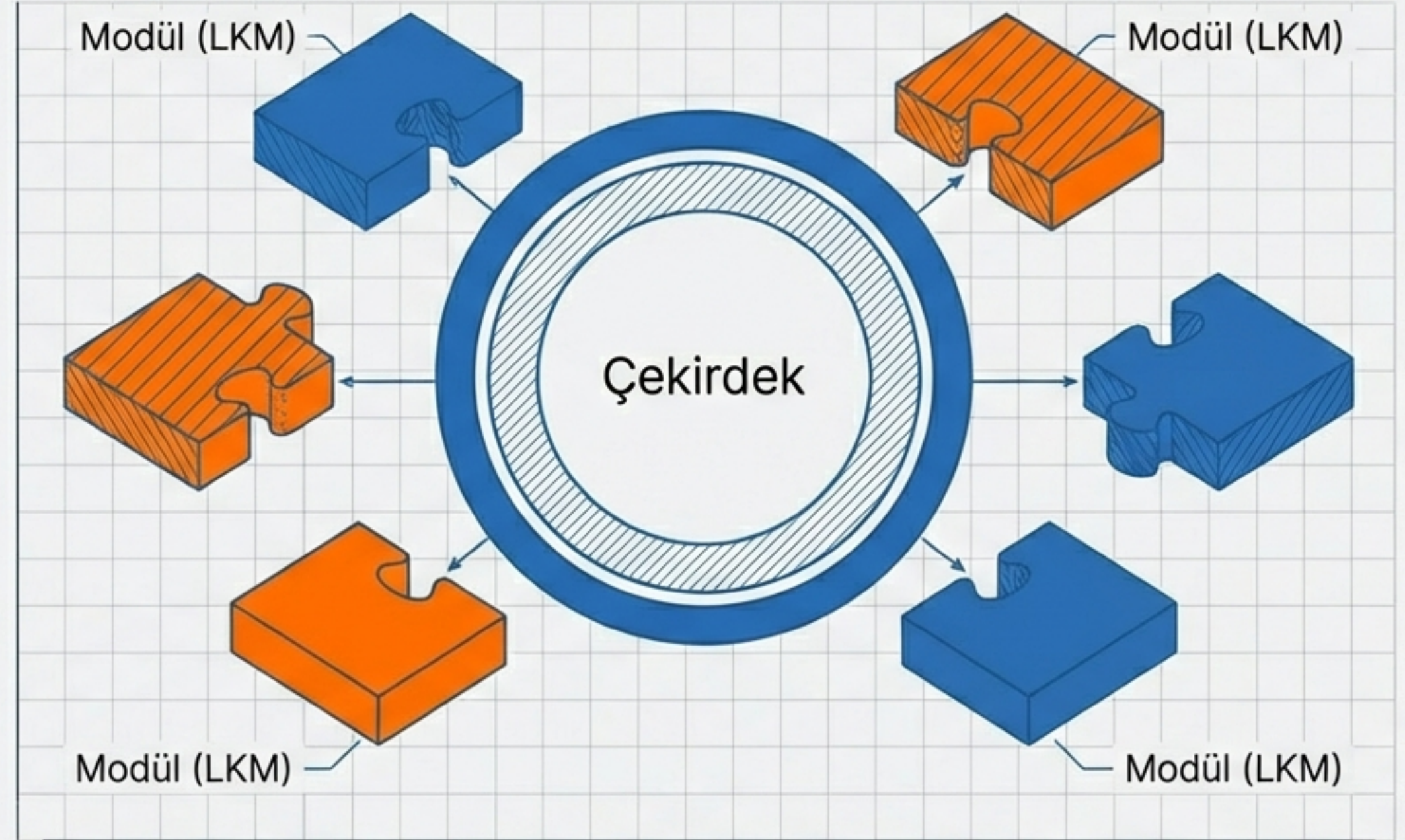
Mikroçekirdek (Microkernel)



↳ Güvenli ve genişletilebilir. İletişim 'Message Passing' ile sağlanır (Mach).

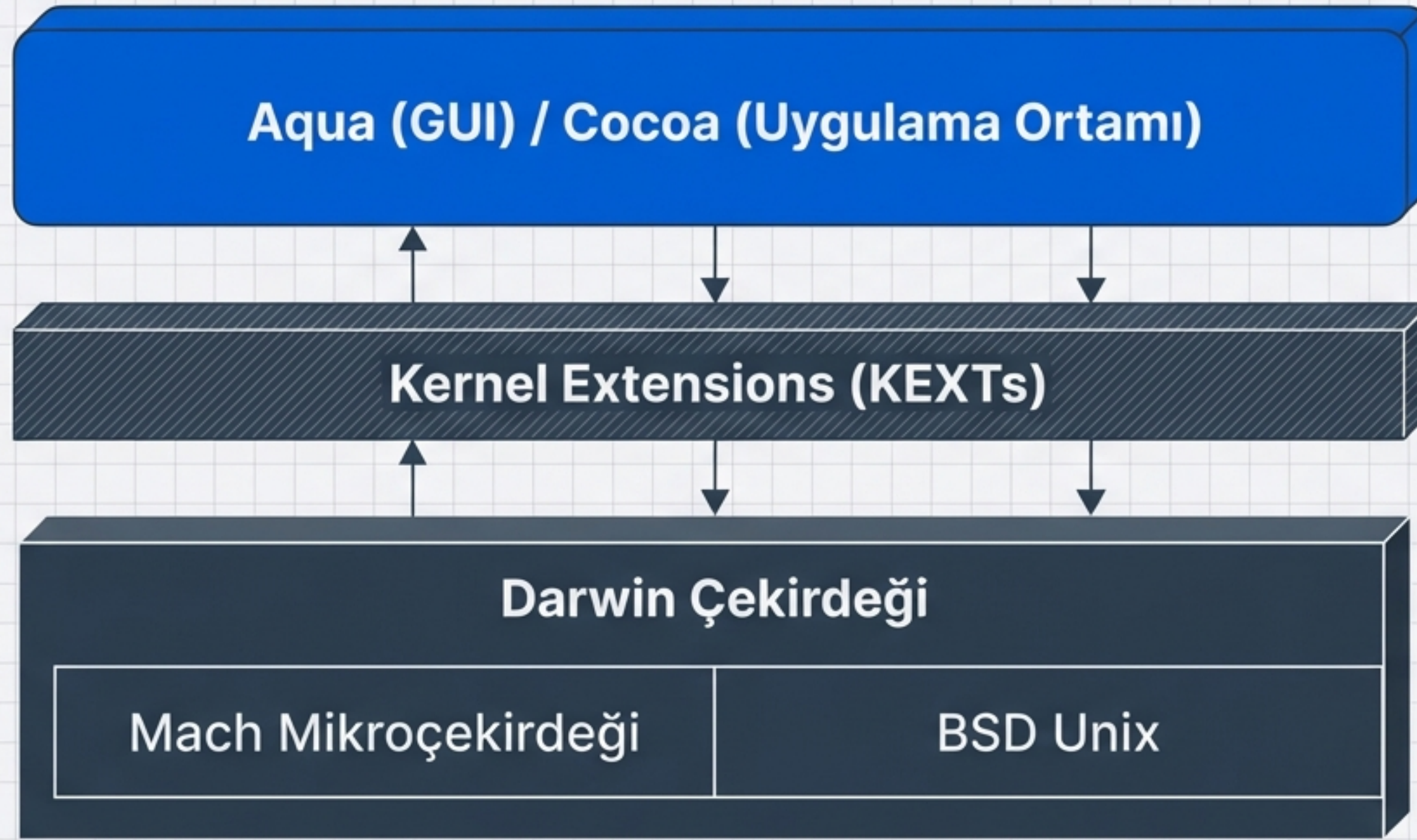
Modern Yaklaşım: Modüller ve Hibrit Sistemler

- **Modüler (LKM):** Nesne yönelimli yaklaşım. İhtiyaç duyulduğunda dinamik yükleme.
- **Hibrit Sistemler:** Monolitik performans + Modüler esneklik.
- **Örnekler:** Linux, Solaris, Windows (Subsystems).



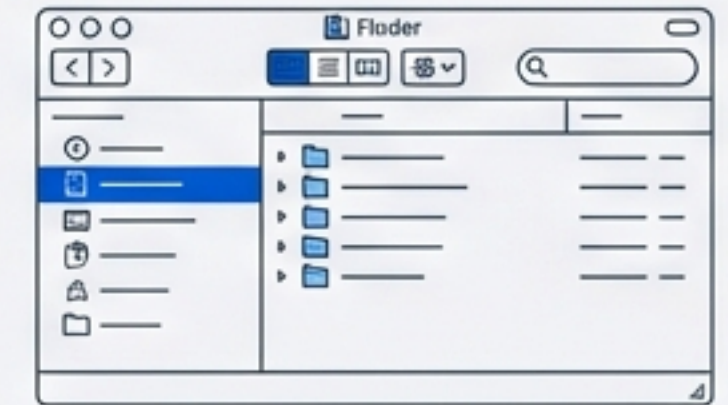
Günümüz sistemleri saf mimariler yerine pratik, hibrit çözümler kullanır.

Vaka Çalışması: Mac OS X ve iOS

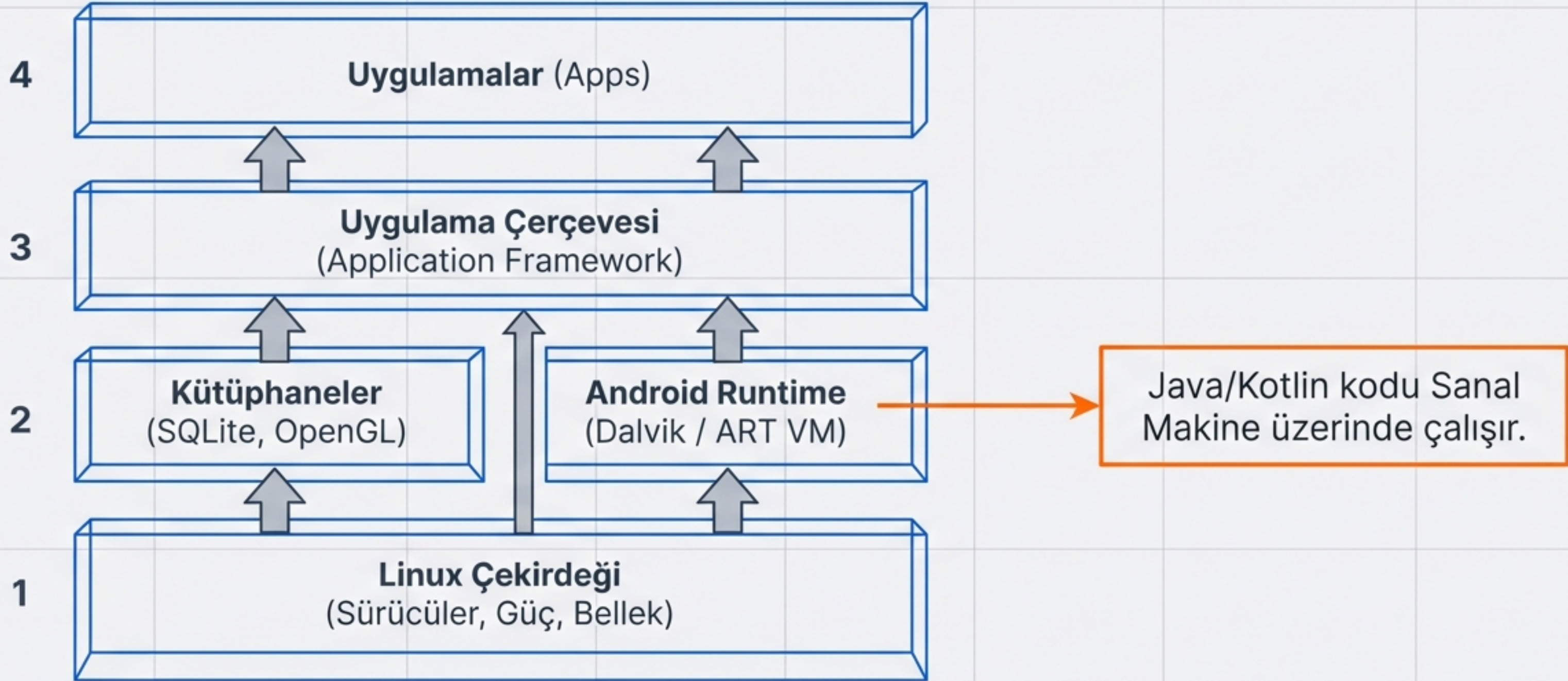


iOS Farkı

- ✂ - ARM Mimarisi
- ✂ - Cocoa Touch (Dokunmatik)
- ✂ - Güç Yönetimi Optimizasyonu



Vaka Çalışması: Android Ekosistemi



iS Hata Ayıklama (Debugging)

“Hata ayıklamak, kodu yazmaktan iki kat zordur. Eğer kodu olabildiğince zekice yazarsanız, hatayı bulacak kadar zeki değilsiniz demektir.””



Core Dump (Uygulama Hatası)

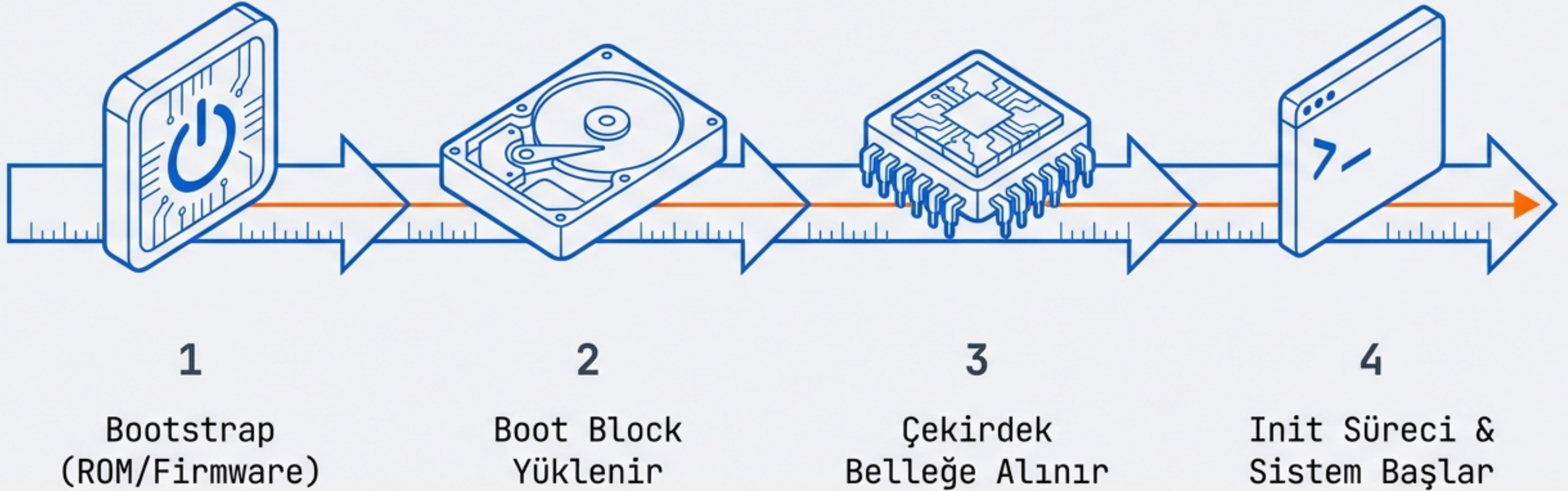


Crash Dump (Çekirdek Hatası)



Profiling (Performans İzleme)

Sistem Önyükleme (System Boot)



Özet: Karmaşıklığı Yönetmek

- İşletim Sistemleri katmanlı bir mimariye sahiptir.
- Sistem Çağrılarını, kullanıcı ile çekirdek arasındaki tek güvenli kapıdır.
- Modern sistemler (Linux, Windows, macOS) performans için hibrit yapıları tercih eder.

